同濟大學

TONGJI UNIVERSITY

实验报告

装

订

线

实验名称	简单神经网络与卷积神经网络
课程名称	深度学习
学　　院	计算机科学与技术学院
专　　业	数据科学与大数据技术
学生姓名	景琛皓
学　　号	2351635
指导教师	程大伟
日　　期	2026.3.18

1 全连接神经网络 (chap4)

1.1 问题描述

利用 numpy 和 tensorflow 、pytorch 搭建全连接神经网络。使用 numpy 实现此练习需要自己手动求导，而 tensorflow 和 pytorch 具有自动求导机制。

1.2 数据集说明

MNIST 数据集包括 60000 张训练图片和 10000 张测试图片。图片样本的数量已经足够训练一个很复杂的模型（例如 CNN 的深层神经网络）。它经常被用来作为一个新的模式识别模型的测试用例。而且它也是一个方便学生和研究者们执行用例的数据集。除此之外，MNIST 数据集是一个相对较小的数据集，可以在笔记本 CPUs 上面直接执行

1.3 NumPy 手动求导实现

在 tutorial_minst_fnn-numpy-exercise.ipynb 中，我们补全了反向传播的关键梯度计算：

矩阵乘法梯度 (Matmul) :

```
def backward(self, grad_y):
        '''
        x: shape(N, d)
        w: shape(d, d')
        grad_y: shape(N, d')
        '''
        x = self.mem['x']
        W = self.mem['W']

        ####################
        '''计算矩阵乘法的对应的梯度'''
        grad_x = np.matmul(grad_y, W.T)
        grad_W = np.matmul(x.T, grad_y)
        ####################
        return grad_x, grad_W
```

根据链式法则，对输入的梯度是输出梯度与权重转置的乘积；对权重的梯度是输入转置与输出梯度的乘积。

ReLU 梯度 :

```
def backward(self, grad_y):
        '''
        grad_y: same shape as x
        '''
        ####################
        '''计算 relu 激活函数对应的梯度'''
        x = self.mem['x']
        grad_x = grad_y * (x > 0)
        ####################
        return grad_x
```

实现了 ReLU 的掩码梯度。只有前向传播中大于 0 的位置，梯度才能通过，这体现了 ReLU 激活函数的非线性门控特性。

1.4 TensorFlow 2.0 自动求导实现

在 tutorial_minst_fnn-tf2.0-exercise.ipynb 中，我们补全了模型结构：

```
class myModel:
    def __init__(self):
        ####################
        '''声明模型对应的参数'''
        self.W1 = tf.Variable(tf.random.normal([784, 256], stddev=0.1))
        self.b1 = tf.Variable(tf.zeros([256]))
        self.W2 = tf.Variable(tf.random.normal([256, 10], stddev=0.1))
        self.b2 = tf.Variable(tf.zeros([10]))
        ####################
    def __call__(self, x):
        ####################
        '''实现模型函数体，返回未归一化的 logits'''
        x = tf.reshape(x, [-1, 784])
        h1 = tf.nn.relu(tf.matmul(x, self.W1) + self.b1)
        logits = tf.matmul(h1, self.W2) + self.b2
        ####################
        return logits
model = myModel()
```

利用 TF2.0 的 tf.Variable 显式声明可训练参数，并通过 tf.nn.relu 构建了一个典型的两层隐藏层网络，为自动求导提供了完整的计算图。

1.5 运行结果

```
epoch 0 : loss 24.72223286454846 ; accuracy 0.07413333333333333
epoch 1 : loss 23.924144333646588 ; accuracy 0.1002
epoch 2 : loss 23.024883096483002 ; accuracy 0.1252
epoch 3 : loss 21.789798024024133 ; accuracy 0.16403333333333334
epoch 4 : loss 20.651041530819974 ; accuracy 0.209
epoch 5 : loss 19.77011018744175 ; accuracy 0.23966666666666667
epoch 6 : loss 18.820253731189734 ; accuracy 0.26993333333333336
epoch 7 : loss 17.89411614473916 ; accuracy 0.3019
epoch 8 : loss 17.241523788018537 ; accuracy 0.32666666666666666
epoch 9 : loss 16.750174736771154 ; accuracy 0.34513333333333335
epoch 10 : loss 16.34424433437691 ; accuracy 0.36118333333333336
epoch 11 : loss 15.995649918471548 ; accuracy 0.37455
epoch 12 : loss 15.686255253464532 ; accuracy 0.38605
epoch 13 : loss 15.398560292015457 ; accuracy 0.39716666666666667
epoch 14 : loss 15.108667168656357 ; accuracy 0.40665
epoch 15 : loss 14.768259637386473 ; accuracy 0.41781666666666667
epoch 16 : loss 14.336060964222801 ; accuracy 0.42795
epoch 17 : loss 13.800404674290487 ; accuracy 0.4438166666666667
epoch 18 : loss 13.238676768175882 ; accuracy 0.45813333333333334
epoch 19 : loss 12.275000859496435 ; accuracy 0.49001666666666666
epoch 20 : loss 11.789863414935745 ; accuracy 0.5096833333333334
epoch 21 : loss 11.099507177500909 ; accuracy 0.5359833333333334
epoch 22 : loss 10.947845925466538 ; accuracy 0.5452
epoch 23 : loss 10.37985058714904 ; accuracy 0.5686333333333333
epoch 24 : loss 10.194234392408456 ; accuracy 0.5764666666666667
...
epoch 47 : loss 5.943588768247914 ; accuracy 0.73995
epoch 48 : loss 5.8297749382365 ; accuracy 0.7436666666666667
epoch 49 : loss 5.800353627628091 ; accuracy 0.7465833333333334
test loss 5.590955869790617 ; accuracy 0.7588
```

图 1.numpy 版本运行结果

CNN_tensorflow.ipynb × tutorial_minst_fnn-tf2.0-exercise.ipynb ×

chap4_ simple neural network > tutorial_minst_fnn-tf2.0-exercise.ipynb > M↓

＋ 代码 ＋ Markdown | 全部运行 清除所有输出 | Outline

```
epoch 1 : loss 0.7520259 ; accuracy 0.83023334
epoch 2 : loss 0.7513737 ; accuracy 0.83031666
epoch 3 : loss 0.75072306 ; accuracy 0.8305
epoch 4 : loss 0.7500742 ; accuracy 0.8305167
epoch 5 : loss 0.74942684 ; accuracy 0.83063334
epoch 6 : loss 0.748781 ; accuracy 0.83071667
epoch 7 : loss 0.74813706 ; accuracy 0.8308
epoch 8 : loss 0.74749464 ; accuracy 0.8308667
epoch 9 : loss 0.74685377 ; accuracy 0.83095
epoch 10 : loss 0.74621457 ; accuracy 0.8311167
epoch 11 : loss 0.74557704 ; accuracy 0.8312167
epoch 12 : loss 0.7449409 ; accuracy 0.83133334
epoch 13 : loss 0.7443065 ; accuracy 0.8314667
epoch 14 : loss 0.7436737 ; accuracy 0.8315833
epoch 15 : loss 0.74304247 ; accuracy 0.83165
epoch 16 : loss 0.74241275 ; accuracy 0.8318167
epoch 17 : loss 0.74178463 ; accuracy 0.8319333
epoch 18 : loss 0.74115807 ; accuracy 0.83208334
epoch 19 : loss 0.7405333 ; accuracy 0.83218336
epoch 20 : loss 0.73990965 ; accuracy 0.8323333
epoch 21 : loss 0.7392879 ; accuracy 0.83241665
epoch 22 : loss 0.7386676 ; accuracy 0.8325833
epoch 23 : loss 0.7380488 ; accuracy 0.8327
epoch 24 : loss 0.73743147 ; accuracy 0.83278334
...
epoch 47 : loss 0.7236447 ; accuracy 0.83501667
epoch 48 : loss 0.72306263 ; accuracy 0.8351167
epoch 49 : loss 0.7224819 ; accuracy 0.83515
test loss 0.69837373 ; accuracy 0.846
```

图 2.tensorflow 版本运行结果

2 函数拟合

2.1 问题描述

根据通用近似定理，一个包含足够多神经元的单隐藏层前馈神经网络（使用非线性激活函数，如 ReLU），可以以任意精度近似闭区间上的任何连续函数。本实验旨在不依赖 PyTorch 或 TensorFlow 等深度学习框架，仅使用 NumPy 库从底层构建一个两层神经网络，拟合一个自定义的复杂非线性函数。

2.2 函数定义与数据工程

目标函数选择 ：定义目标函数为 $y = \sin(x) + 0.5x$ 。

该函数结合了周期性的正弦波动和线性的增长趋势。正弦部分测试了模型捕捉高频非线性变化的能力，线性部分则测试了模型在大尺度范围内的稳定性。

数据采集策略 ：

- 在区间 $[-\pi, \pi]$ 内均匀采样 1000 个点作为训练样本，确保模型能够学习到函数完整的周期特性。
- 另选 200 个独立的采样点作为测试集，用于验证模型的泛化能力。
- 标准化处理 ：虽然本实验数据规模较小，但在输入网络前确保了数据维度的严格对齐（均为 (N, 1) 结构），以适配矩阵运算。。

2.3 模型架构与数学实现 (NumPy 底层逻辑)

本模型采用了 1-64-1 的三层架构（含输入输出层）：

隐藏层设计 ：包含 64 个神经元。选择 ReLU ($f(z) = \max(0, z)$) 作为激活函数。ReLU 的优势在于其导数在正半轴恒为 1，能有效缓解深层网络中的梯度消失问题，且计算效率极高。

参数初始化：

权重初始化为 np.random.randn(in, out) * np.sqrt(2. / in) 。

理论依据 ：针对 ReLU 激活函数，这种初始化方式能保持每一层输出的方差一致，避免模型在训练初期陷入饱和区或梯度爆炸。

反向传播推导 ：

损失函数采用 MSE ：$L = \Sigma (y_{pred} - y_{true})^2 / m$。

梯度计算公式：

- 输出层误差： $\delta_2 = 2(y_{pred} - y_{true})/m$
- 隐藏层误差： $\delta_1 = (\delta_2 \cdot W_2^T) \odot f(z_1)$ ，其中 f' 为 ReLU 的导数。

2.4 实验过程与参数调优

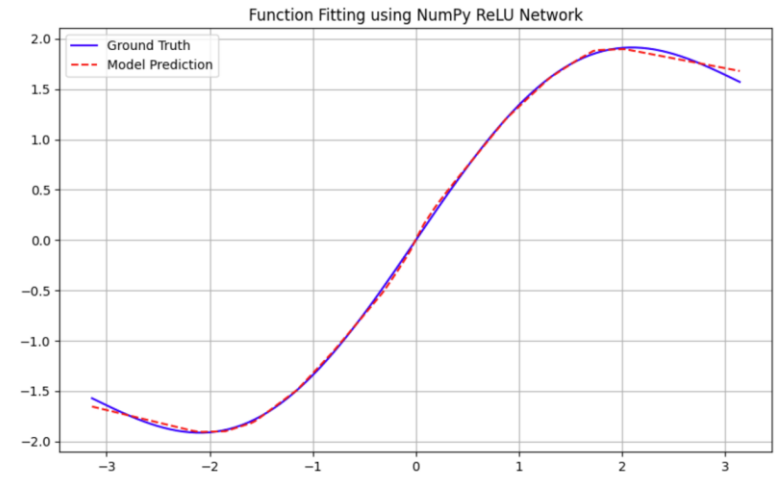
在实验过程中，我经历了一次关键的调优过程：

1. 初期失效分析 ：初始设置 lr = 0.1 时，模型出现了 RuntimeWarning: overflow 和 Loss = nan 。

由于 NumPy 实现中没有内置的梯度裁剪（Gradient Clipping），过大的学习率导致权重更新步长过大，使激活值超出了浮点数表示范围（梯度爆炸）。

2. 优化策略 ：
   - 将学习率降低至 0.01 ，以换取训练的稳定性。
   - 将训练轮数增加至 20,000 轮 ，弥补低学习率带来的收敛速度减慢。
   - 结果记录 ：Loss 从初始的 1.63 稳步下降，并在 15,000 轮后进入平台期。

2.5 实验结果与性能评估



Final Test MSE Loss: 0.000909

图 3.函数拟合实验运行结果

定量评估 ：最终测试集 MSE 降至 0.000909 ，达到了极高的拟合精度。

定性分析 ：观察拟合图像可以看到，红色预测曲线不仅完美覆盖了波峰和波谷，而且在函数转折处的衔接非常自然。

ReLU 局部线性特征 ：放大图像可以观察到预测曲线实际上是由 64 段微小的直线拼接而成的，这直观地展示了神经网络如何通过分段线性函数组合来逼近复杂的连续曲线。

实验证明，通过精心设计的参数初始化和合理的超参数配置，仅使用基础的 NumPy 矩阵运算也能构建出表达能力强大的深度学习模型。

3 卷积神经网络(CNN)

3.1 问题描述

利用卷积神经网络，实现对 MNIST 数据集的分类问题。

3.2 数据集说明

MNIST 数据集包括 60000 张训练图片和 10000 张测试图片。图片样本的数量已经足够训练一个很复杂的模型（例如 CNN 的深层神经网络）。它经常被用来作为一个新的模式识别模型的测试用例。而且它也是一个方便学生和研究者们执行用例的数据集。除此之外，MNIST 数据集是一个相对较小的数据集，可以在笔记本 CPUs 上面直接执行

3.3 TensorFlow 卷积层补全

在 CNN_tensorflow.ipynb 中，我们实现了卷积核和池化的具体操作。

核心函数修改 ：

```
def conv2d(x, W):
    # 每一维度  滑动步长全部是 1， padding 方式 选择 same
    # 提示 使用函数  tf.nn.conv2d
    return tf.nn.conv2d(x, W, strides=[1, 1, 1, 1], padding='SAME')
def max_pool_2x2(x):
    # 滑动步长 是 2 步; 池化窗口的尺度 高和宽度都是 2; padding 方式 请选择 same
    # 提示 使用函数  tf.nn.max_pool
    return tf.nn.max_pool(x, ksize=[1, 2, 2, 1], strides=[1, 2, 2, 1], padding='SAME')
```

这里我们严格遵循 README 要求，卷积步长设为 1，池化步长设为 2 且窗口为 2x2，使用 SAME 填充以保证输出尺寸可控。

卷积层参数修改 ：

- W_conv1 : 设置为 [7, 7, 1, 32] ，对应 README 的 7x7 补全要求。
- W_conv2 : 设置为 [5, 5, 32, 64] ，对应 5x5 的深层特征提取要求。

3.4 PyTorch 卷积层补全

在 CNN_pytorch.ipynb 中，我们利用 nn.Sequential 模块化构建网络。

Conv2d 参数修改 ：

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d( # ???
                # patch 7 * 7 ; 1  in channels ; 32 out channels ; ; stride is 1
                # padding style is same
                in_channels=1      ,
                out_channels=32     ,
                kernel_size=7      ,
                stride=1           ,
                padding=3          ,
            ),
            nn.ReLU(),        # activation function
            nn.MaxPool2d(2),  # pooling operation
        )
        self.conv2 = nn.Sequential( # ???
            # line 1 : convolution function, patch 5*5 , 32 in channels ;64 out channels; padding
style is same; stride is 1
            # line 2 : choosing your activation funciont
            # line 3 : pooling operation function.
            nn.Conv2d(32, 64, 5, 1, 2),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.out1 = nn.Linear( 7*7*64 , 1024 , bias= True)   # full connection layer one
        self.dropout = nn.Dropout(keep_prob_rate)
        self.out2 = nn.Linear(1024,10,bias=True)
```

计算并设置了正确的 padding （如 7 × 7 核需 padding=3 ）以实现 SAME 效果。

维度转换修改 ：

```
def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = x.view(x.size(0), -1)
```

将四维的特征图张量（Batch, Channel, H, W）安全地拉伸为二维矩阵，适配后续全连接层的输入。

3.5 运行结果

```
0.088
0.855
0.908
0.932
0.941
0.947
0.951
0.958
0.968
0.969
0.972
0.972
0.977
0.977
0.98
0.981
0.981
0.981
0.983
0.981
```

图 4.pytorch 版本运行结果

```
========== 20 ===== ===== test accuracy is  0.342 ==========
========== 40 ===== ===== test accuracy is  0.366 ==========
========== 60 ===== ===== test accuracy is  0.464 ==========
========== 80 ===== ===== test accuracy is  0.56 ==========
========== 100 ===== ===== test accuracy is  0.624 ==========
========== 120 ===== ===== test accuracy is  0.694 ==========
========== 140 ===== ===== test accuracy is  0.716 ==========
========== 160 ===== ===== test accuracy is  0.758 ==========
========== 180 ===== ===== test accuracy is  0.778 ==========
========== 200 ===== ===== test accuracy is  0.794 ==========
========== 220 ===== ===== test accuracy is  0.826 ==========
========== 240 ===== ===== test accuracy is  0.85 ==========
========== 260 ===== ===== test accuracy is  0.844 ==========
========== 280 ===== ===== test accuracy is  0.87 ==========
========== 300 ===== ===== test accuracy is  0.856 ==========
========== 320 ===== ===== test accuracy is  0.862 ==========
========== 340 ===== ===== test accuracy is  0.872 ==========
========== 360 ===== ===== test accuracy is  0.876 ==========
========== 380 ===== ===== test accuracy is  0.892 ==========
========== 400 ===== ===== test accuracy is  0.91 ==========
========== 420 ===== ===== test accuracy is  0.884 ==========
========== 440 ===== ===== test accuracy is  0.896 ==========
========== 460 ===== ===== test accuracy is  0.898 ==========
========== 480 ===== ===== test accuracy is  0.912 ==========
========== 500 ===== ===== test accuracy is  0.892 ==========
...
========== 1120 ===== ===== test accuracy is  0.974 ==========
========== 1140 ===== ===== test accuracy is  0.976 ==========
========== 1160 ===== ===== test accuracy is  0.978 ==========
========== 1180 ===== ===== test accuracy is  0.976 ==========
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

图 5.tensorflow 版本运行结果